



## *Artificial Intelligence*

Lecture Note: icyice

Illustration: ©SEGA ©Colorful Palette

©Crypton Future Media

## Contents

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>1</b> | <b>Problem Solving</b>                    | <b>1</b>  |
| §1.1     | Search Problem . . . . .                  | 1         |
| §1.2     | Adversarial Problem . . . . .             | 6         |
| §1.3     | Constraint Satisfaction Problem . . . . . | 10        |
| <b>2</b> | <b>Knowledge, Reasoning and Planning</b>  | <b>12</b> |
| §2.1     | Knowledge and Reasoning . . . . .         | 12        |
| §2.2     | Probabilistic Reasoning . . . . .         | 16        |
| §2.3     | Bayesian Network . . . . .                | 18        |
| §2.4     | Markov Network . . . . .                  | 23        |
| <b>3</b> | <b>Machine Learning</b>                   | <b>26</b> |
| §3.1     | Learning for PGM . . . . .                | 26        |
| §3.2     | Artificial Neural Network . . . . .       | 28        |
| §3.3     | Deep Learning . . . . .                   | 30        |
| §3.4     | Markov Decision Process . . . . .         | 31        |

# 1 Problem Solving

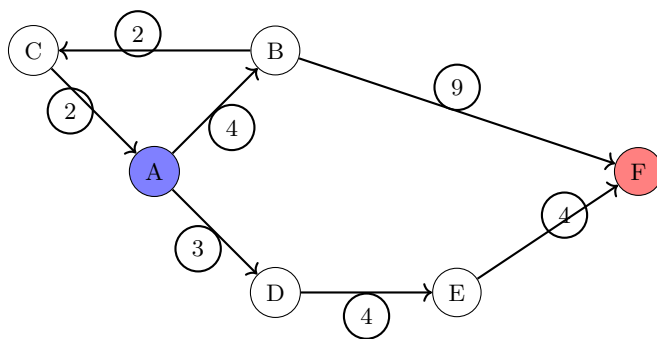
## §1.1 Search Problem

A search problem consists of:

- States
- Initial state
- Goal test
- Actions
- Transition model
- Path cost



### Example 1.1



- State space:
  - Vertices
- Initial state:
  - A
- Goal test:
  - Is state = F?
- Actions:
  - Go to adjacent vertex with cost = weight
- Transition model
- Path cost
  - cost = weight
- Solution: A → D → E → F

### Definition 1.1 State Graphs

State space graph: A mathematical representation of a search problem



- **Nodes** are (abstracted) world configurations(**states**)
- **Arcs** represent successors (action results)
- **The goal test** is a set of goal nodes (maybe only one)

**Notation:**

In a state graph, each state occurs only once.

**Definition 1.2 Search Tree**

A "what if" tree of plans and their outcomes.



- The start state is the root node
- Children nodes correspond to successors
- Nodes show states, but correspond to PLANS that achieve those states

**Notation:**

For most of the problems, we can never build the whole tree. The search tree does not remember whether the node has occurred or not, so it may have infinite outcomes.

To avoid redundant paths, we could mark the node that has occurred (because the following tree is repeated).

**Summary:****Algorithm 1 Tree Search**

- 1: **function** TREE-SEARCH(problem) **returns** a solution, or failure
- 2: initialize the frontier using the initial state of problem
- 3: loop do
- 4:   **if** the frontier is empty **then return** failure
- 5:   choose a leaf node and remove it from the frontier
- 6:   **if** the node contains a goal state **then return** the corresponding solution
- 7:   expand the chosen node, adding the resulting nodes to the frontier

**Algorithm 2 Graph Search**

- 1: **function** GRAPH-SEARCH(problem) **returns** a solution, or failure
- 2: initialize the frontier using the initial state of problem
- 3: initialize the explored set to be empty
- 4: loop do
- 5:   **if** the frontier is empty **then return** failure
- 6:   choose a leaf node and remove it from the frontier
- 7:   **if** the node contains a goal state **then return** the corresponding solution
- 8:   add the node to the explored set
- 9:   expand the chosen node, adding the resulting nodes to the frontier
- 10:   only if not in the frontier or explored set

**Annotation:**

Some important ideas:

- **Frontier**—Nodes to be explore

- **Expansion**—Add the children nodes to the current node
- **Explored set**—The set of nodes that have been expanded
- **Exploration strategy**—which leaf nodes to explore first

Performance measurements:

- **Completeness**: Guaranteed to find a solution if one exists?
- **Optimality**: Guaranteed to find the optimal solution?
- **Time complexity**: How long?
- **Space complexity**: How much memory?



#### Annotation:

Search strategies:

- **Uninformed search**(blind search): no additional information about states
- **Informed search**(heuristic search): knows how “promising” a non-goal state is

### Definition 1.3 Breadth First Search

- **Strategy**:
  - Expand a shallowest node first
- **Implementation**:
  - Frontier is a **FIFO** (first in first out) queue



#### Notation:

BFS will expand the search tree layer by layer, so for the worst conditions, the time and space complexities are both  $O(b^d)$ .



#### Annotation:

How to optimize breadth first search:

- **Cost-Sensitive Search**
  - To find **the optimal solution** with any step cost function
- **Strategy**:
  - Expand a **cheapest** node first
- **Implementation**:
  - Frontier is a **priority queue**

### Definition 1.4 Depth First Search

- **Strategy:**
  - Expand a deepest node first
- **Implementation:**
  - Frontier is a LIFO (last in first out) stack



#### Notation:

If the depth is infinite, DFS might not find the solution, but for tree search, the space complexity is  $O(bm)$  (put every children node into the stack).

Iterative Deepening DFS:

- **Idea:** get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution...
  - Run a DFS with depth limit 2. If no solution...
  - ...
  - Run a DFS with depth limit  $d$ . ...



#### Annotation:

Most of the problem works in the low levels, so not so bad to be redundant.

### Definition 1.5 Cost-sensitive Search

- **Strategy:**
  - Expand a cheapest node first
- **Implementation:**
  - Frontier is a priority queue



#### Summary:

Uninformed Search Strategies:

| Criterion | Breadth-First    | Uniform-Cost                          | Depth-First | Depth-Limited | Iterative Deepening |
|-----------|------------------|---------------------------------------|-------------|---------------|---------------------|
| Complete? | Yes <sup>a</sup> | Yes <sup>a,b</sup>                    | No          | No            | Yes <sup>a</sup>    |
| Time      | $O(b^d)$         | $O(b^{1+\lceil C^*/\epsilon \rceil})$ | $O(b^m)$    | $O(b^l)$      | $O(b^d)$            |
| Space     | $O(b^d)$         | $O(b^{1+\lceil C^*/\epsilon \rceil})$ | $O(bm)$     | $O(bl)$       | $O(bd)$             |
| Optimal?  | Yes <sup>c</sup> | Yes                                   | No          | No            | Yes <sup>c</sup>    |

If no any other information is offered, there's no search algorithm that is not exponential.

**Definition 1.6 Search Heuristics****A heuristic function is:**

- A function that estimates how close a state is to a goal
- Designed for a particular search problem
- Examples: Manhattan distance, Euclidean distance for pathing

**Definition 1.7 Greedy Best-First Search****Strategy:**

- Expand a node with lowest  $h(n)$  first

**Notation:**

Greedy Best-First Search is incomplete, if  $h(n)$  is smaller but there is no solution in that path, the algorithm will end up in loops.

**Definition 1.8 A\* Search****Strategy:**

- Let  $g(n)$  be cost incurred already on path to  $n$
- Expand nodes with lowest  $g(n) + h(n)$  first



If the heuristic is **admissible**, the **tree-search** version of A\* is optimal

- A heuristic  $h(n)$  is **admissible** if

$$0 \leq h(n) \leq h^*(n)$$

where  $h^*(n)$  is the true cost to a nearest goal

- Example: straight-line distance is admissible
- Admissible heuristic means that A\* is always optimistic

**Notation:**

For graph search, the heuristic needs to be consistent.

A heuristic  $h(n)$  is **consistent (monotonicity)** if for every node  $n$  and every successor  $n'$  of  $n$  generated by any action  $a$ ,

$$h(n) \leq c(n, a, n') + h(n')$$

It is a form of the general triangle inequality.

**Annotation:**

- If the heuristic is **admissible**, the **tree-search** version of A\* is optimal
- If the heuristic is **consistent**, the **graph-search** version of A\* is optimal

Which means A\* is complete and optimally efficient.



### Example 1.2

Design a heuristic function for the puzzle below

- **Initial state:**

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 |   | 4 |
| 7 | 6 | 5 |

- **Goal state:**

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |

solve. There are two common solutions.

- $h_1$  = the number of tiles misplaced
- $h_2$  = the sum of distances of the tiles from their goal positions

## §1.2 Adversarial Problem

### Definition1.9 Adversarial Search Problem

A problem is an adversarial search problem if it has:

- Multi-agent environment
- Competitive environment



Adversarial problem has many forms, but we mainly talk about Zero-sum Games.

### Definition1.10 Zero-sum Game

- Zero-Sum Games
  - Agents have **opposite** utilities
  - Pure competition:
    - \* One **maximizes**, the other **minimizes**



### Annotation:

A game is standard, if it is:

- deterministic
- observable
- two-player
- turn-taking
- zero-sum



A search problem consists of

- $S_0$ : the initial state
- **Player**(s): defines which player has the move in a state
- **Actions**(s): returns the set of legal moves in a state
- **Result**(s, a): the transition model
- **Terminal-test**(s): is true when the game is over and false otherwise
- **Utility**(s, p): a utility / objective function

The initial state, actions, and results define the game tree.

### Definition 1.11 Minimax Search

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if Terminal-Test}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$



#### Notation:

The minimax searches from the leaf nodes to the root nodes (reversed algorithm).

### Algorithm 3 Minimax Decision Algorithm

- 1: **function** MINIMAX-DECISION(s) **returns** an action
- 2:   **return** the action  $a \in \text{Actions}(s)$  with the highest Minimax-Value(Result(s, a))
- 3: **function** MINIMAX-VALUE(s) **returns** a value
- 4:   **if** Terminal-Test(s) **then return** Utility(s)
- 5:   **if** Player(s) = MAX **then return**  $\max_{a \in \text{Actions}(s)} \text{Minimax-Value}(\text{Result}(s, a))$
- 6:   **if** Player(s) = MIN **then return**  $\min_{a \in \text{Actions}(s)} \text{Minimax-Value}(\text{Result}(s, a))$

If the players are more than 2, we can use vectors to demonstrate the value of each node.



#### Summary:

##### Generalization of minimax:

- Terminals have **utility tuples**
- Node values are also utility tuples
- **Each player maximizes its own component**
- Can give rise to cooperation and competition dynamically...

In all, the minimax algorithm is DFS, so it has the same performance of DFS.



#### Annotation:

A Go game has  $10^{360}$  possible states, which means even if we compute it from the moment the universe forms,

we are still computing it!

This is why we need more efficient algorithm.

### Definition1.12 Alpha-Beta Pruning

- $\alpha$  = the highest value **found till now** for MAX on path to root
- $\beta$  = the lowest value **found till now** for MIN on path to root



General principles:

- Consider a **node n**;
- If MAX has a better choice  $\alpha$  either at the parent node of **n** or at any choice point further up, then **n** will never be reached in actual play.
- So once we found out enough about **n** (by examining some of its descendants) to reach this conclusion, we can prune it.



### Notation:

If  $\alpha \geq \beta$ , then we do not need to search the path.

### Definition1.13 Evaluation Functions

**An evaluation function** returns an estimate of the expected utility of the game from a given position.



In practice, we typically weighed linear sum of features.

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$



### Annotation:

To improve the efficient, we could use more tricks.

- **Quiescent search**: terminate search only in **quiescent positions**, i.e., no major changes expected in feature values
- **Forward pruning**: beam search, consider only a “beam” of the  $n$  best moves
- **Table lookup**: keep a table that summarizes the expertise of humans or experience from previous games

### Definition1.14 Monte Carlo Tree Search

**MCTS** builds a statistics tree that partially maps onto the game tree



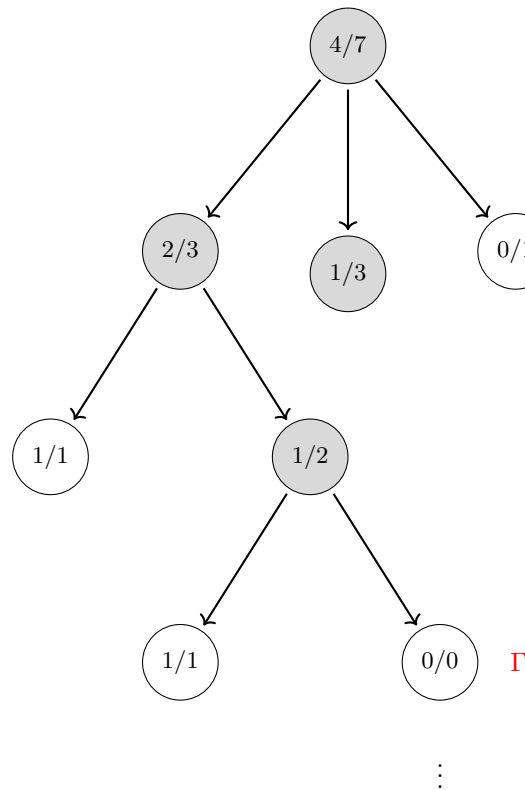
Basic steps:

- **Selection**: Starting at root node R, recursively select optimal child nodes until a leaf node L is reached.
- **Expansion**: If L is not a terminal node then create one or more child nodes and select one C.
- **Simulation**: Run a simulated playout from C until a result is achieved.

- Backpropagation: Update the current move sequence with the simulation result.



### Example 1.3



1. **Node Labeling:**  $w/n$  ( $w$  = wins from node,  $n$  = total visits)
2. **Tree Snapshot:** After 7 MCTS iterations (root:  $4/7 = 4$  wins / 7 visits).
  - Gray nodes: Explored (visited 1); White nodes: Under-explored/leaf.
  - Red  $\Gamma$ : Pending node (0/0, next to simulate).
3. **Core MCTS Cycle (4 Steps):**
  1. Selection: UCT formula to pick optimal path (balance exploit/explore)
  2. Expansion: Add child nodes to leaf/pending nodes
  3. Simulation: Random payout to game end (win/loss/draw)
  4. Backpropagation: Update  $w/n$  for all nodes in the path

## §1.3 Constraint Satisfaction Problem

### Definition1.15 Constraint Satisfaction Problem

A constraint satisfaction problem consists of three components,  $X$ ,  $D$ , and  $C$ :

- $X$  is a set of **variables**,  $\{X_1, \dots, X_n\}$ .
- $D$  is a set of **domains**,  $\{D_1, \dots, D_n\}$ , one for each variable.
- $C$  is a set of **constraints** that specify allowable combinations of values.



It can be helpful to visualize a CSP as a constraint graph.



#### Annotation:

There some properties of variables and constraints:

#### Variables:

- Discrete variables(finite or infinite)
- Continuous variables

#### Constraints:

- Unary constraints
- Binary constraints
- Higher-order constraints

Since this is a search problem, we could use the search algorithms we introduced before. However, their efficiency is terrible. We need more algorithms.

### Definition1.16 Backtracking Search for CSPs

- **One variable at a time:** Only need to consider assignments to a single variable at each step.
- **Check the constraints as we go:** Consider values that do not conflict with previous assignments.



How could we improve it?

### Definition1.17 Variable and Value Ordering

#### Minimum-remaining-value heuristic

- Choose the variable with the fewest "legal" values(**most constrained variable** or **fail-first heuristic**).

#### Degree heuristic

- Choose the variable with the largest degree.

#### Least-constraining heuristic

- Leave the maximum flexibility of the remaining variables.



#### Notation:

The idea is to get backtrack as soon as possible, avoiding unnecessary search.

**Definition 1.18 Arc Consistency**

An arc  $X \rightarrow Y$  is consistent if and only if for every  $x$  there is some  $y$  which could be assigned without violating a constraint.

**Notation:**

Arc consistency **can not detect all the failures**, so it still needs backtracking.

**Definition 1.19 K-Consistency**

For every  $(k - 1)$  variables, we could always find the  $k$ th variable that satisfies the constraints.



Strong K-consistency:

- Also  $(k - 1), (k - 2) \dots 2, 1$  consistent.

**Annotation:**

Strong K-consistency means we can solve the problem without backtracking.

To even improve the efficiency, we could decompose the major problem into independent subproblems and convert it into a tree-structured CSP.

**Summary:**

Nearly Tree-Structured CSP:

1. **Removing nodes:** Construct a cutset that leaves the remaining constraint graph a tree.
2. **Collapsing nodes together:** Solve the constraints connecting between subproblems using the tree methods.

If this looks complicated to you, that is just because it IS.

## 2 Knowledge, Reasoning and Planning

### §2.1 Knowledge and Reasoning

#### Definition 2.1 Logic

- **Logics**
  - Formal languages for representing information such that conclusions can be drawn
- **Syntax**
  - Defines the sentences in the language
- **Semantics**
  - Defines the truth of a sentence in a possible world (model)



#### Example 2.1

- $x + 2 \geq y$  is a sentence;  $x2 + y >$  is not a sentence
- $x + 2 \geq y$  is true iff the number  $x + 2$  is no less than the number  $y$
- $x + 2 \geq y$  is true in a world where  $x = 7, y = 1$
- $x + 2 \geq y$  is false in a world where  $x = 0, y = 6$

#### Definition 2.2 Models

- **Models** are mathematical abstractions, each of which simply fixes the truth or falsehood of every relevant sentences.
- $m$  is a model of a sentence  $\alpha$  if  $\alpha$  is true in model  $m$ .
- $M(\alpha)$  : the set of all models of  $\alpha$ .



#### Definition 2.3 Entailment

Entailment: a sentence follows logically from another:

$$KB \models \alpha$$

Knowledge base  $KB$  entails sentence  $\alpha$  iff  $\alpha$  is true in all models where  $KB$  is true

$$KB \models \alpha \text{ if and only if } M(KB) \subseteq M(\alpha)$$

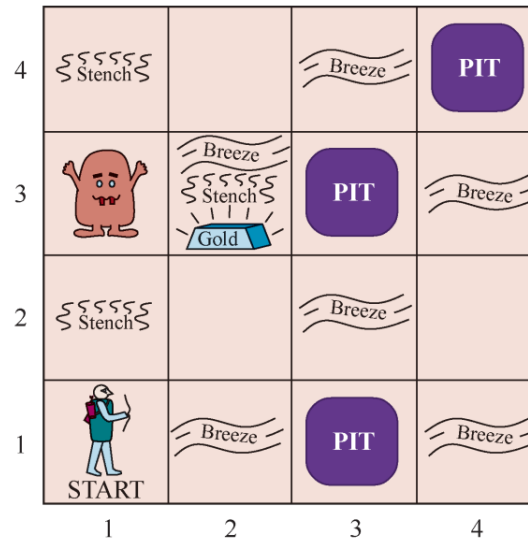


Now we consider a boring but simple game - The Wumpus World.



#### Example 2.2

The Wumpus World.



- **Performance measure**

- gold +1000, death -1000
- -1 per step, -10 for using the arrow

- **Environment**

- squares adjacent to wumpus are stench
- squares adjacent to pit are breezy
- glitter iff gold is in the same square
- shooting kills wumpus if you are facing it
- shooting uses up the only arrow
- grabbing picks up gold if in same square
- releasing drops the gold in same square

- **Actuators**

- Left turn, Right turn, Forward, Grab, Shoot, Climb (only at (1,1))

- **Sensors**

- Stench, Breeze, Glitter, Bump(to the wall), Scream(if the Wumpus gets shoot)

#### Definition 2.4 Inference

If an inference algorithm  $i$  can derive  $\alpha$  from  $KB$

$$KB \vdash_i \alpha$$



We usually consider two properties of inference.

- **Soundness**

- $i$  is sound if
- whenever  $KB \vdash_i \alpha$ , it is also true that  $KB \models \alpha$

- **Completeness**

- $i$  is complete if

- whenever  $KB \models \alpha$ , it is also true that  $KB \vdash_i \alpha$



### Annotation:

Like *Discrete Math*, we introduce some basic logical connectives.

- If  $P$  is a sentence,  $\neg P$  is a sentence (negation)
- If  $P_1$  and  $P_2$  are sentences,  $P_1 \wedge P_2$  is a sentence (conjunction)
- If  $P_1$  and  $P_2$  are sentences,  $P_1 \vee P_2$  is a sentence (disjunction)
- If  $P_1$  and  $P_2$  are sentences,  $P_1 \implies P_2$  is a sentence (implication)
- If  $P_1$  and  $P_2$  are sentences,  $P_1 \iff P_2$  is a sentence (biconditional)

Let's take a look back at [The Wumpus World](#).

- Let  $P_{i,j}$  be true if there is a pit in  $[i, j]$ 
  - observation  $R_1 : \neg P_{1,1}$
- Let  $B_{i,j}$  be true if there is a breeze in  $[i, j]$ .
- “Pits cause breezes in adjacent squares”
  - rule  $R_2 : B_{1,1} \iff (P_{1,2} \vee P_{2,1})$
  - rule  $R_3 : B_{2,1} \iff (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
  - observation  $R_4 : \neg B_{1,1}$
  - observation  $R_5 : B_{2,1}$

### Definition 2.5 Logical Equivalence

Two sentences are logically equivalent iff true in same models:

$$\alpha \equiv \beta \text{ if and only if } \alpha \models \beta \text{ and } \beta \models \alpha$$



For more, please reference to *Discrete Math* course.

- **Validity**
  - A sentence is valid if it is true in **all** models,  
e.g.,  $\text{True}, A \vee \neg A, A \implies A, (A \wedge (A \implies B)) \implies B$
  - Validity is connected to inference via the Deduction Theorem:  
 $KB \models \alpha$  if and only if  $(KB \implies \alpha)$  is valid
- **Satisfiability**
  - A sentence is satisfiable if it is true in **some** model  
e.g.,  $A \vee B, C$
  - A sentence is unsatisfiable if it is true in **no** models  
e.g.,  $A \wedge \neg A$
  - Satisfiability is connected to inference via the following:  
 $KB \models \alpha$  if and only if  $(KB \wedge \neg \alpha)$  is unsatisfiable



**Theorem 2.1 Proof by Resolution**

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where  $\ell_i$  and  $m_j$  are complementary literals.



Now let's redesign the logic in *The Wumpus World* into Conjunctive Normal Form.

$$B_{1,1} \iff (P_{1,2} \vee P_{2,1})$$

1. Eliminate  $\iff$ , replacing  $\alpha \iff \beta$  with  $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$ .

$$(B_{1,1} \implies (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \implies B_{1,1})$$

2. Eliminate  $\implies$ , replacing  $\alpha \implies \beta$  with  $\neg\alpha \vee \beta$ .

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move  $\neg$  inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ( $\vee$  over  $\wedge$ ) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

**Notation:**

In theory, we can redesign every logical sentence into conjunctive normal form or disjunctive normal form, but it may make the sentence unreasonably long (up to  $2^n$ ).

Every definite clause can be written as

- a conjunction of positive literals  $\implies$  a single positive literal

Samples from *The Wumpus World*.

$$\neg L_{1,1} \vee \neg Breeze \vee B_{1,1} \quad (L_{1,1} \wedge Breeze) \implies B_{1,1}$$

**Summary:**

The algorithm for Proof by Resolution.

**Algorithm 4** PL-RESOLUTION Algorithm

```

1: function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
2:   inputs:  $KB$ , the knowledge base, a sentence in propositional logic
3:            $\alpha$ , the query, a sentence in propositional logic
4:    $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg\alpha$ 
5:    $new \leftarrow \{\}$ 
6:   loop do
7:     for each pair of clauses  $C_i, C_j$  in  $clauses$  do
8:        $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
9:       if  $resolvents$  contains the empty clause then return true
10:       $new \leftarrow new \cup resolvents$ 
11:   if  $new \subseteq clauses$  then return false
12:    $clauses \leftarrow clauses \cup new$ 

```

**Definition2.6** First-Order Logic

The world contains objects, relations and functions.



## §2.2 Probabilistic Reasoning

**Annotation:**

This subsection will use the knowledge from *Probability and Mathematical Statistics* and *Stochastic Process*.

**Definition2.7** Random Variable

A random variable is some aspect of the world about which we have uncertainty.

**Definition2.8** Probability Distribution

Associate a probability with each value

$$\forall x \quad P(X = x) \geq 0, \quad \text{and} \quad \sum_x P(X = x) = 1$$



More definitions like Probability Density Function and Joint Probability Distribution could be seen in my *Probability and Mathematical Statistics* note.

**Definition2.9** Event

An **event** is a set **E** of outcomes

$$P(E) = \sum_{(x_1, \dots, x_n) \in E} P(x_1, \dots, x_n)$$



For most of the conditions, we could use **normalization** to calculate probability.

$$\begin{aligned} P(W = s \mid T = c) &= \frac{P(W = s, T = c)}{P(T = c)} \\ &= \frac{P(T = c, W = s)}{\sum_W P(T = c, W)} \end{aligned}$$

### Definition 2.10 Enumeration

General case:

- Evidence variables:  $E_1 \cdots E_k = e_1 \cdots e_k$
- Query variable:  $Q$
- Hidden variables:  $H_1 \cdots H_k$



### Notation:

We use upper case to represent random variables and lower case to represent the samples.

### Theorem 2.2 The Chain Rule

The joint distribution can be written as an incremental product of conditional distributions:

$$P(x_1, x_2, x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$

$$P(x_1, x_2, \dots, x_n) = \prod P(x_i|x_1 \dots x_{i-1})$$



From the chain rule, we could have one of the most important formula in AI algorithm.

### Theorem 2.3 Bayes' Rule

$$P(x, y) = P(y)P(x|y) = P(x)P(y|x) \Rightarrow P(x|y) = \frac{P(y|x)}{P(y)} P(x)$$



Let's take a look back at The Wumpus World.

- We know the following facts:
  - $b = \neg b_{1,1} \wedge b_{1,2} \wedge b_{2,1}$
  - $\text{known} = \neg p_{1,1} \wedge \neg p_{1,2} \wedge \neg p_{2,1}$
- Query is  $\mathbf{P}(P_{1,3}|\text{known}, b)$
- Define  $\text{Unknown} = P_{ij}$ s other than  $P_{1,3}$  and  $\text{Known}$
- For inference by enumeration, we have

$$\mathbf{P}(P_{1,3}|\text{known}, b) = \alpha \sum_{\text{unknown}} \mathbf{P}(P_{1,3}, \text{unknown}, \text{known}, b)$$

But calculate it will be exponential, that's why we introduced some optimization.

$$\begin{aligned} \mathbf{P}(P_{1,3}|\text{known}, b) &= \alpha \sum_{\text{unknown}} \mathbf{P}(P_{1,3}, \text{unknown}, \text{known}, b) \\ &= \alpha' \mathbf{P}(P_{1,3}) \sum_{\text{fringe}} \mathbf{P}(b|\text{known}, P_{1,3}, \text{fringe}) \mathbf{P}(\text{fringe}) \end{aligned}$$

Now we know that the blocks far away from the block we are interested in will not influence the result.

## §2.3 Bayesian Network

### Definition 2.11 Bayesian Network

**Bayesian Networks:** a technique for describing complex joint distributions (models) using simple, local distributions (conditional probability)

A Bayesian network is

- A **directed acyclic graph (DAG)**
- Each node corresponds to a random variable  $X_i$
- Each node  $X_i$  has a **conditional probability distribution (CPD)**

$$P(X_i | \text{Parents}(X_i))$$



#### Annotation:

The BN represents a **joint distribution** via the chain rule for Bayesian networks

$$P(X_1, X_2, \dots, X_n) = \prod_i P(X_i | \text{Parents}(X_i))$$

Assume conditional independent:

$$P(X_i | X_1, \dots, X_{i-1}) = P(X_i | \text{Parents}(X_i))$$

### Definition 2.12 Conditional Independence

Two variables are **independent** ( $X \perp Y$ ) in a joint distribution if:

- $P(X, Y) = P(X)P(Y)$
- $\forall x, y \quad P(x, y) = P(x)P(y)$
- $P(x|y) = P(x)$
- $P(y|x) = P(y)$

So, how could we infer two variables are conditionally independent?

**Theorem 2.4 Markov Blanket**

A node is conditionally independent of all other nodes in the network, given its parents, children and children's parents

**Definition 2.13 Active Trails**

A trail  $X_1 - X_2 - \dots - X_n$  is active if it has no v-structures  $X_{i-1} \rightarrow X_i \leftarrow X_{i+1}$

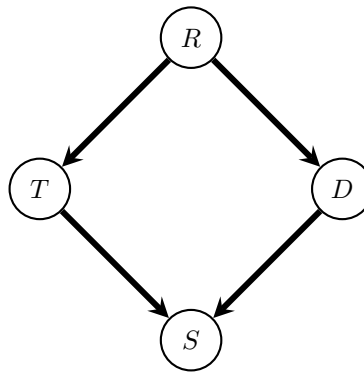
**Summary:**

d-Separation: Two sets of nodes  $X$  and  $Y$  are d-separated by a set of evidence nodes  $Z$  if and only if  $Z$  blocks every undirected path between any node in  $X$  and any node in  $Y$ .

When  $X$  and  $Y$  are d-separated, we can tell that  $X$  and  $Y$  are conditionally independent.

**Example 2.3**

Determine when  $T$  and  $D$  are conditionally independent.



**Keys:** When  $R$  is observed and  $S$  is not observed.

**Notation:**

Variables Elimination Ordering:

- The computational and space complexity of variable elimination is determined by the largest factor
- The elimination ordering can greatly affect the size of the largest factor.

**Definition 2.14 Sampling**

- Draw  $N$  samples from a sampling distribution  $S$
- Compute an approximate posterior probability
- Show this converges to the true probability  $P$

**Prior Sampling**

- This process generates samples with probability:

$$S_{PS}(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{Par}(X_i)) = P(x_1, \dots, x_n)$$

- Let the number of samples of an event be  $N_{PS}(x_1, \dots, x_n)$
- Then

$$\begin{aligned} \lim_{N \rightarrow \infty} \hat{P}(x_1, \dots, x_n) &= \lim_{N \rightarrow \infty} N_{PS}(x_1, \dots, x_n) / N \\ &= S_{PS}(x_1, \dots, x_n) \\ &= P(x_1, \dots, x_n) \end{aligned}$$

### Likelihood Weighting

- Sampling distribution if  $\mathbf{z}$  sampled and  $\mathbf{e}$  fixed evidence

$$S_{WS}(\mathbf{z}, \mathbf{e}) = \prod_{i=1}^l P(z_i | \text{Parents}(Z_i))$$

- Now, samples have weights

$$w(\mathbf{z}, \mathbf{e}) = \prod_{i=1}^m P(e_i | \text{Parents}(E_i))$$

- Together, weighted sampling distribution is consistent

$$S_{WS}(\mathbf{z}, \mathbf{e}) \cdot w(\mathbf{z}, \mathbf{e}) = \prod_{i=1}^l P(z_i | \text{Parents}(z_i)) \prod_{i=1}^m P(e_i | \text{Parents}(e_i)) = P(\mathbf{z}, \mathbf{e})$$

So yeah, if this looks complicated to you, that is just it IS.

### Definition 2.15 Transition Model

The transition model specifies the probability distribution of the current state given previous models

$$P(X_t | X_{0:t-1})$$



### Annotation:

To simplify the problem, we usually assume the current state depends on only **a finite fixed number of** previous states, which is called Markov Process.

For example, the first-order Markov chain:

$$P(X_t | X_{0:t-1}) = P(X_t | X_{t-1})$$

A Markov dynamic system is stationary (time invariance).

**Definition 2.16 Observation Model**

The probability distribution of the current evidence given the state sequence and the previous observation

$$P(E_t | X_{0:t}, E_{0:t-1})$$

**Summary:**

Use the Markov assumption, we could tell a generic temporal model:

- The transition model:

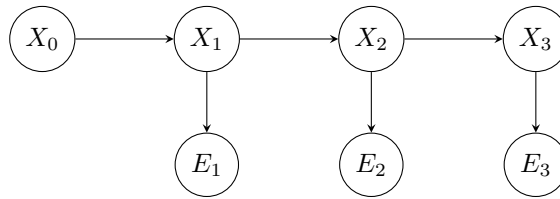
$$P(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = P(\mathbf{X}_t | \mathbf{X}_{t-1})$$

- The sensor/observation model:

$$P(\mathbf{E}_t | \mathbf{X}_{0:t}, \mathbf{E}_{0:t-1}) = P(\mathbf{E}_t | \mathbf{X}_t)$$

- The initial state model:  $P(\mathbf{X}_0)$
- Joint distribution:

$$P(\mathbf{X}_{0:t}, \mathbf{E}_{1:t}) = P(\mathbf{X}_0) \prod_{i=1}^t P(\mathbf{X}_i | \mathbf{X}_{i-1}) P(\mathbf{E}_i | \mathbf{X}_i)$$

**Filtering**

- **Goal:** Compute  $P(X_t | e_{1:t})$
- Recursive estimation:

$$\begin{aligned} \mathbf{P}(\mathbf{X}_{t+1} | \mathbf{e}_{1:t+1}) &= f(\mathbf{e}_{t+1}, \mathbf{P}(\mathbf{X}_t | \mathbf{e}_{1:t})) \\ &= \alpha \underbrace{P(e_{t+1} | X_{t+1})}_{\text{Update}} \underbrace{P(X_{t+1} | e_{1:t})}_{\text{Prediction}} \end{aligned}$$

**Prediction**

- Compute  $P(X_{t+k} | e_{1:t})$
- Viewed as **filtering** without the addition of new evidence.
- Recursively computed by

$$\mathbf{P}(\mathbf{X}_{t+k+1} | \mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+k}} \underbrace{\mathbf{P}(\mathbf{X}_{t+k+1} | \mathbf{x}_{t+k})}_{\text{All Possible States}} P(\mathbf{x}_{t+k} | \mathbf{e}_{1:t})$$

**Smoothing**

- Compute  $P(X_k | e_{1:t})$   $0 \leq k < t$
- Smoothing provides a better estimate of the state than was available at the time

- Learning requires smoothing, rather than filtering
- Forward-backward algorithm

$$\begin{aligned}
 P(\mathbf{X}_k \mid \mathbf{e}_{1:t}) &= P(\mathbf{X}_k \mid \mathbf{e}_{1:k}, \mathbf{e}_{k+1:t}) \\
 &= \alpha P(\mathbf{X}_k \mid \mathbf{e}_{1:k}) P(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k, \mathbf{e}_{1:k}) \\
 &= \alpha \underbrace{P(\mathbf{X}_k \mid \mathbf{e}_{1:k})}_{\text{Forward}} \underbrace{P(\mathbf{e}_{k+1:t} \mid \mathbf{X}_k)}_{\text{Backward}}
 \end{aligned}$$

### The Most Likely Sequence

- $\text{argmax}_{X_{1:t}} P(X_{1:t} \mid \mathbf{e}_{1:t})$
- Viterbi algorithm

$$\max_{\mathbf{x}_1 \dots \mathbf{x}_t} P(\mathbf{x}_1, \dots, \mathbf{x}_t, \mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) = \alpha P(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \max_{\mathbf{x}_t} \left( P(\mathbf{X}_{t+1} \mid \mathbf{x}_t) \max_{\mathbf{x}_1 \dots \mathbf{x}_{t-1}} P(\mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{x}_t \mid \mathbf{e}_{1:t}) \right)$$



### Notation:

If you can't remember the formula... Well, you can infer them from the Bayes' Formula, but it might take a while to calculate.

### Definition 2.17 Hidden Markov Models

HMM is a state-observation model in which the state is described by a single discrete random variable. ♣

### Definition 2.18 Dynamic Bayesian Networks

Dynamic Bayesian Network (DBN) represents a temporal probability model in which each slice can have any number of state variables and evidence variables ♣

### Algorithm 5 Particle Filtering Algorithm

```

1: function PARTICLE-FILTERING( $e, N, dbn$ ) returns a set of samples for the next time step
2:   inputs:  $e$ , the new incoming evidence
3:            $N$ , the number of samples to be maintained
4:            $dbn$ , a DBN with prior  $P(\mathbf{X}_0)$ , transition model  $P(\mathbf{X}_1|\mathbf{X}_0)$ , sensor model  $P(\mathbf{E}_1|\mathbf{X}_1)$ 
5: persistent:  $S$ , a vector of samples of size  $N$ , initially generated from  $P(\mathbf{X}_0)$ 
6: local variables:  $W$ , a vector of weights of size  $N$ 
7: for  $i = 1$  to  $N$  do
8:    $S[i] \leftarrow$  sample from  $P(\mathbf{X}_1|\mathbf{X}_0 = S[i])$  {step 1}
9:    $W[i] \leftarrow P(e|\mathbf{X}_1 = S[i])$  {step 2}
10: end for
11:  $S \leftarrow$  WEIGHTED-SAMPLE-WITH-REPLACEMENT( $N, S, W$ ) {step 3}
12: return  $S$ 

```

If you can't understand and this looks complicated to you, well, me the same.



## §2.4 Markov Network

### Definition 2.19 Markov Network

A Markov network is

- An undirected graph whose nodes are random variables  $X_1, \dots, X_n$
- Factors (potential) over given scopes  $\Phi(D_k), k = 1, \dots, K$ , represented by a clique



The MN represents a joint distribution

$$P(X_1, \dots, X_n) = \frac{1}{Z} \prod_c \Phi_c(X_c)$$



#### Notation:

The factors (potential functions) have no direct relation with the probability, which we introduced normalization factor  $Z$  to make it the probability.

### Definition 2.20 Gibbs Distribution

A distribution  $P_\Phi$  is a **Gibbs distribution** parameterized by a set of factors  $\Phi = \{\phi_1(D_1), \dots, \phi_k(D_k)\}$  if it is defined as follows:

$$P_\Phi(X_1, \dots, X_n) = \frac{1}{Z} \tilde{P}_\Phi(X_1, \dots, X_n)$$



So in Bayesian Networks we talked about conditional Independence, what about Markov Network?

### Theorem 2.5 U-Separation

$X$  and  $Y$  are separated in  $H$  given  $Z$  if there is no active trail in  $H$  between  $X$  and  $Y$  given  $Z$ , denoted as  $sep_H(X, Y \mid Z)$ .



#### Annotation:

In brief, if we remove the vertices in  $Z$ ,  $X$  and  $Y$  are not connected anymore, then  $X$  and  $Y$  are conditional independence.

### Definition 2.21 Boltzmann Machine

A fully connected graph with pairwise potentials on binary-valued nodes (for  $X_i \in \{-1, +1\}$ , or,  $X_i \in \{0, 1\}$ ) is called a **Boltzmann machine**

$$P(X) = \frac{1}{Z} \exp \left( \sum_{ij} \phi_{ij}(x_i, x_j) \right)$$



**Definition 2.22 Conditional Random Fields**

Given observed variables  $X$  and target variables  $Y$ , CRF describes the **conditional distribution**

$$P(Y|X) = \frac{1}{Z(X)} P(X, Y) = \frac{1}{Z(X)} \prod_i \Phi_i(D_i)$$

$$Z(X) = \sum_Y P(X, Y)$$



CRF doesn't assume that observations are independent.

**Definition 2.23 Cluster Graph**

A **cluster graph** is an undirected graph such that

- Nodes are clusters  $C_i \subseteq \{X_1, \dots, X_n\}$
- Edge between  $C_i$  and  $C_j$  associated with **sepset**  $S_{i,j} \subseteq C_i \cap C_j$



- Given set of factors  $\Phi$ , we assign each  $\Phi_k$  to a cluster  $C_{\alpha(k)}$  s.t.

$$\text{Scope}(\Phi_k) \subseteq C_{\alpha(k)}$$

- For each factor  $\Phi_k \in \Phi$ , there exists a cluster  $C_i$  s.t.  $\text{Scope}(\Phi_k) \subseteq C_i$

So how could we construct a cluster graph?

- For each  $\Phi_k \in \Phi$ , a factor cluster  $C_k = \text{Scope}(\Phi_k)$
- For each  $X_i$  a singleton cluster  $\{X_i\}$
- Edge  $C_k - X_i$  if  $X_i \in C_k$

**Definition 2.24 Message Passing**

Message passing from cluster  $C_i$  to cluster  $C_j$

$$\delta_{i \rightarrow j}(S_{i,j}) = \sum_{C_i - S_{i,j}} \psi_i \times \prod_{k \in (N_i - \{j\})} \delta_{k \rightarrow i}$$

**Summary:**

Belief Propagation Algorithm:

- Step 1: Construct a cluster graph
  - Assign each factor  $\Phi_k$  to a cluster  $C_{\alpha(k)}$
  - Construct initial potentials  $\psi_k(C_k) = \prod_{\Phi_i \in C_k} \Phi_i$
- Step 2: Initialize all messages to be 1
- Step 3: **Repeat**

- Select edge  $(i, j)$  and pass message

$$\delta_{i \rightarrow j}(S_{i,j}) = \sum_{C_i - S_{i,j}} \psi_i \times \prod_{k \in (N_i - \{j\})} \delta_{k \rightarrow i}$$

- Step 4: Compute **belief**

$$\beta_i(C_i) = \psi_i \times \prod_{k \in N_i} \delta_{k \rightarrow i}$$

A **cluster graph** is calibrated if every pair of adjacent clusters  $C_i$  and  $C_j$  agree on their sepset  $S_{i,j}$

$$\sum_{C_i - S_{i,j}} \beta_i(C_i) = \sum_{C_j - S_{i,j}} \beta_j(C_j)$$



#### Notation:

If the graph contains loops, then the algorithm might not converge, thus it will have pseudo-marginal.

If you can't understand this subsection, you can just skip it. The exam will not cover it.

### 3 Machine Learning

#### §3.1 Learning for PGM

##### Definition 3.1 KL-Divergence

$$\mathbf{D}(P^* \parallel \tilde{P}) = \mathbf{E}_{\xi \sim P^*} \left[ \log \left( \frac{P^*(\xi)}{\tilde{P}(\xi)} \right) \right]$$



##### Notation:

If  $P^* = 0$ , then we say  $0 \log \frac{0}{\cdot} = 0$ .

If  $P^* \neq 0, \tilde{P} = 0$ , then the KL-Divergence is infinite.

##### Definition 3.2 Likelihood

$$P(\mathcal{D} : \mathcal{M}) = \prod_{m=1}^M P(\xi[m] : \mathcal{M})$$



To better calculate it, we usually use its logarithm form.

$$\log P(\mathcal{D} : \mathcal{M}) = \sum_{m=1}^M \log P(\xi[m] : \mathcal{M})$$

In *Probability and Mathematical Statistics*, we have learned the Maximum Likelihood Estimation, take coins for example.

- Find  $\theta$  maximizing likelihood

$$L(\theta : \mathcal{D}) = \theta^{M[1]}(1 - \theta)^{M[0]}$$

- Equivalent to maximizing log-likelihood

$$\ell(\theta : \mathcal{D}) = M[1] \log \theta + M[0] \log(1 - \theta)$$

- Differentiating the log-likelihood and solving for  $\theta$

$$\hat{\theta} = \frac{M[1]}{M[1] + M[0]}$$



##### Annotation:

In BN network, we can derive MLE

$$L(\theta | D) = \prod_{j=1}^N P(d_j | \theta) = \prod_{j=1}^N \prod_i P(x_i^{(j)} | pa(x_i)^{(j)}, \theta_i)$$

Where  $pa(x_i)$  are parents nodes.

But in some cases, MLE will derive 0 outcome, so we need another prediction.

### Theorem 3.1 Bayesian Prediction

$$\begin{aligned} P(x[M+1] | x[1], \dots, x[M]) \\ = \int P(x[M+1] | \theta) \frac{P(x[1], \dots, x[M] | \theta) P(\theta)}{P(x[1], \dots, x[M])} d\theta \end{aligned}$$



For biased coins:

$$\begin{aligned} P(X[M+1] = x^1 | x[1], \dots, x[M]) &= \frac{1}{P(x[1], \dots, x[M])} \int \theta \cdot \theta^{M[1]} (1 - \theta)^{M[0]} d\theta \\ &= \frac{M[1] + 1}{M[1] + M[0] + 2} \end{aligned}$$



### Annotation:

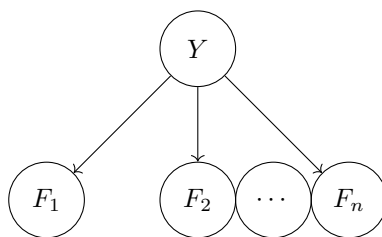
Beta Distribution:

$$f(x; \alpha, \beta) = \frac{x^{\alpha-1} (1-x)^{\beta-1}}{B(\alpha, \beta)}$$

$$E(X) = \frac{\alpha}{\alpha + \beta}$$

$$Var(X) = \frac{\alpha\beta}{(\alpha + \beta)^2 (\alpha + \beta + 1)}$$

Naive Bayesian Model:



- Independence:

$$\forall F_i, F_j, \quad (F_i \perp F_j | Y)$$

- Joint distribution:

$$P(Y, F_1, \dots, F_n) = P(Y) \prod_i P(F_i | Y)$$

- Classifier:

$$y^* = \operatorname{argmax} P(Y = y_j | f_1, \dots, f_n) = \operatorname{argmax} \frac{P(Y = y_j) \prod_i P(f_i | Y = y_j)}{P(f_1, \dots, f_n)}$$

**Notation:**

We assume that all nodes are conditionally independent upon giving label, and it's not ideal in real life. But in most of the cases, it's still good enough.

For small dataset, we need to smooth it.

**Definition3.3 Laplace Smoothing**

$$P_{LAP}(x) = \frac{c(x) + 1}{\sum_x [c(x) + 1]} = \frac{c(x) + 1}{N + |X|}$$



Laplace Smoothing performs well in uniform distribution, but poorly in practice, especially in conditional distribution. We need a new method.

**Definition3.4 Linear Interpolation**

$$P_{LIN}(x | y) = \alpha P(x | y) + (1.0 - \alpha)P(x)$$



For those who have hidden variables, we could use Expectation Maximization(EM) algorithm.

- E-Step (Expectation): Complete the data using current parameters
- M-Step (Maximization): Estimate parameters relative to data completion, using [MLE](#)

## §3.2 Artificial Neural Network

If we could use simple models, then no complex model is needed.

- Simpler hypotheses tend to generalize to future data better
- [Prefer the simplest hypothesis consistent with data](#)

**Notation:**

Everything has its cost, we need to choose the model that fits the problem.

**Definition3.5 Decision Tree**

- [Input](#): a vector of attribute values
- [Output](#): a single output value
- Reaches its decision by performing a sequence of tests
- Each node corresponds to a test of the value of one of the input attributes



Choose the one that maximizes the [information gain](#): the expected reduction in entropy

$$\text{Gain}(A) = B \left( \frac{p}{p+n} \right) - \text{Remainder}(A)$$

- The entropy of a Boolean random variable:

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

- The expected entropy remaining after testing A

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

### Definition 3.6 Perceptron

The classic perceptron is a linear binary classifier that maps an input vector  $\mathbf{x}$  to an output  $f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ , where  $\mathbf{w}$  is the weight vector,  $b$  is the bias, and  $\text{sign}(z)$  returns +1 if  $z > 0$  and -1 otherwise. ♣



#### Notation:

The classic perceptron can only handle linear problem, for non-linear problem like XOR function, there's no solution.

#### Activation function:

- The purpose of activation functions is to introduce nonlinearities into the network
- All activation functions are non-linear

So how could we optimize loss?



#### Summary:

##### Gradient Decent:

1. Initialize weights randomly  $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient,  $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
4. Update weights,  $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
5. Return weights

But in some cases, calculate gradient is not that easy, we need a new strategy.

### Theorem 3.2 Backpropagation

Also known as chain rule.

$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$



You can prove this formula by using the chain rule of partial derivative.

### §3.3 Deep Learning

#### Definition 3.7 2D Discrete Convolution

For two discrete two-dimensional signals  $f$  and  $g$ , their 2D discrete convolution  $(f * g)[m, n]$  is defined as:

$$(f * g)[m, n] = \sum_i \sum_j f[i, j] \cdot g[m - i, n - j]$$

where the sums are taken over all indices where the terms are defined (typically with zero padding outside the boundaries).

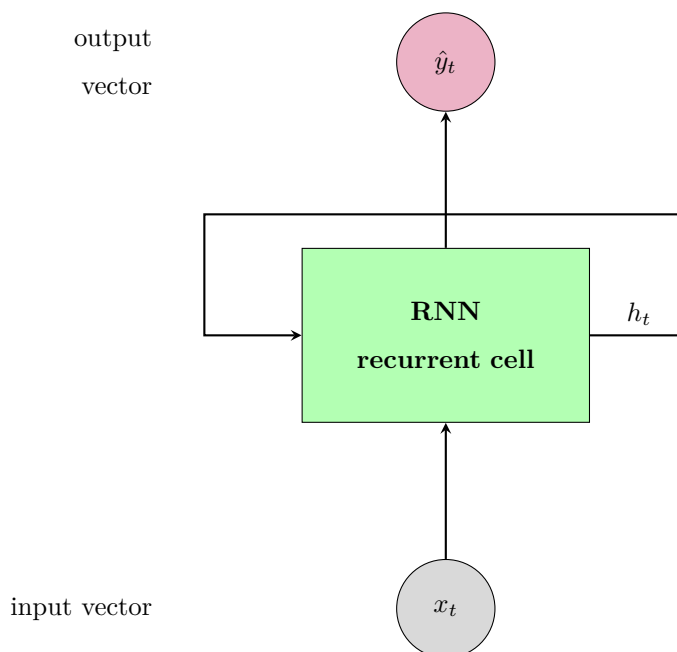


#### Annotation:

If input is  $H_1 \times W_1$ , kernel is  $H_2 \times W_2$ , pad  $p$ , stride  $s$ , then the outcome is

$$\begin{cases} H_{out} = \left\lfloor \frac{H_1 + 2p - H_2}{s} \right\rfloor + 1 \\ W_{out} = \left\lfloor \frac{W_1 + 2p - W_2}{s} \right\rfloor + 1 \end{cases}$$

One example is Recurrent Neural Network(RNN):



To calculate, we need some formulas,

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$



### Definition 3.8 Sigmoid Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$



In Long Short Term Memory (LSTM) model, it has 3 gates.

- Gate1: Forget irrelevant information

$$f_t = \sigma(W_i[h_{t-1}, x_t] + b_f)$$

- Gate2: Identify new information to be stored

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- Gate3: Output filtered version of cell state

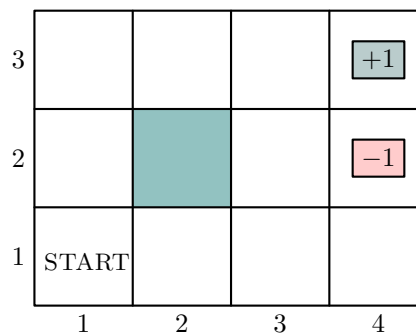
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

## §3.4 Markov Decision Process

An **Markov Decision Process (MDP)** is defined by:

- A set of **states**  $s \in S$
- A set of **actions**  $a \in A$
- A **transition function**  $T(s, a, s')$ 
  - Probability from  $s$  leads to  $s'$ , i.e.  $P(s'|s, a)$
  - Also called the model or the dynamics
- A **reward function**  $R(s, a, s')$ 
  - Sometimes just  $R(s)$  or  $R(s')$
- A **start state**
- Maybe a **terminal state**



In deterministic single-agent search problems, we wanted an optimal **plan**, or sequence of actions, from start to a

goal.

For MDPs, we want an **optimal policy**  $\pi^* : S \rightarrow A$



### Annotation:

In this course, we mainly consider infinite horizon, which makes the optimal policy stationary.

### Theorem 3.3 Assumption on the Utilities

If we assume **stationary preferences**:

$$[s_1, s_2, \dots] \succ [s'_1, s'_2, \dots]$$

$$\Updownarrow$$

$$[s_0, s_1, s_2, \dots] \succ [s_0, s'_1, s'_2, \dots]$$

Then, there are only two ways to define utilities

- **Additive rewards:**

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

- **Discounted rewards:**

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$



With discounted rewards, the utility of an infinite sequence is finite.

$$U_h([s_0, s_1, s_2, \dots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = R_{\max} / (1 - \gamma)$$

To calculate the value of each state, we can use recursive:

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



### Notation:

Though it will converge, the speed is unbelievably slow.

If you don't trust me, you could calculate an MDP by hand, and you will cry.

To improve speed, we introduce Policy Iteration.

**Evaluation:** For fixed current policy  $\pi$ , find values with policy evaluation

- Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

**Improvement:** For fixed values, get a better policy using policy extraction

- One-step look-ahead

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

First we fix the policy, then we maximize the policy.

You don't need to know why we can compute this. You only need to know how to calculate.



### Summary:

This course is over till this page. Artificial Intelligence(ISEE) mainly focuses on basic and traditional AI algorithms, but as you know, AI grows rapidly recently. If you want to gain a deep insight of AI, please abandon this note and look up to recent essays.

## Final Review

### Search

- Uninformed Search
  - Breadth First Search
  - Depth First Search
  - Iterative Deepening Search
  - Uniform-cost Search
- Informed Search (Heuristic Search)
  - Greedy Search
  - A\* Search

### Adversarial Search

- Minimax Search
- Alpha Beta Pruning

### Constraint Satisfaction Problem

- A CPS consists of three components:
  - A set of variables  $X = \{X_1, \dots, X_n\}$   
e.g.,  $X = \{WA, NT, SA, Q, NSW, V, T\}$
  - A set of domains  $D = \{D_1, \dots, D_n\}$   
e.g.,  $D = \{r, g, b\}$
  - A set of constraints  $C$   
e.g. adjacent states must have different colors
- Backtracking search
- How to improve backtracking: Arc-consistency

### Knowledge and Reasoning

- Knowledge-based Agent:
 

A knowledge-based agent is composed of a knowledge base and an inference mechanism.
- Propositional Logic
- First-Order Logic
- Knowledge Representation
- Inference methods:
 

model checking and proof

## Probability

- Conditional probability  $P(x|y) = \frac{P(x,y)}{P(y)}$
- Product rule  $P(x,y) = P(x|y)P(y)$
- Chain rule

$$P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2) \cdots$$

$$= \prod_{i=1}^n P(X_i|X_1, \dots, X_{i-1})$$

- Bays' Rule  $P(x|y) = \frac{P(y|x)}{P(y)} P(x)$
- X, Y independent if and only if:  $\forall x, y : P(x, y) = P(x)P(y)$
- X and Y are conditionally independent given Z if and only if:

## Bayesian Network

- Representation
  - Bayesian network
  - Temporal models
- Inference
  - Exact inference
    - \* Enumeration
    - \* Variable elimination
  - Approximate inference
    - \* Sampling
- Applications
  - Kalman filtering
  - Particle filtering
  - Object tracking

## Markov Network

- Representation
- Inference
  - Exact inference
    - \* Belief propagation
    - \* Max-Sum elimination
  - Approximate inference
    - \* Loopy belief propagation
    - \* Variational methods

## Learning for PGM

- Parameter Estimation
  - Maximum Likelihood Estimation
  - Bayesian Estimation
  - Application: Supervised Classification
- Structure Learning
  - Learning Metrics
- Learning with Hidden variables
  - Expectation Maximization
  - Application: Unsupervised Clustering

## Learning

- Supervised learning
  - Classification
  - Regression
- Machine learning methods
  - Decision Trees
  - Perceptron
  - Artificial Neural Networks
- Convolutional Neural Networks
- Recurrent Neural Networks
- Autoencoder
- Generative Adversarial Networks

## Markov Decision Process

- Problem Formulation
- Value Iteration
- Policy Iteration

If you want to get a better grade, doing some papers is much more useful than watching this note.

听懂的同学举个手……好的，我们继续。——yqq